

高切坡agent开发--阶段总结一

项目阶段性总结

项目定位

当前已实现能力

1. Text-to-SQL
2. 网页端展示
3. 领域知识注入
4. SQL 自纠错能力
5. 真实数据库对象探测
6. 调试与可解释性

当前系统结构

当前已知限制

1. 真实数据库与设计文档不完全一致
2. 复杂业务问题仍可能失败
3. 预警类字段取值仍有不确定性
4. 缺少系统化测试
5. 仍缺少成熟的失败分级与降级响应

当前阶段结论

下一阶段建议

1. 增强 agent 的查询与纠错能力
2. 建设 Schema RAG 知识库
3. 建设 Domain RAG 领域知识库
4. 让 agent 能结合查询结果做专业分析
5. 建立标准化测试集与评估体系
6. 最后再推进生产化能力

结语

项目阶段性总结

项目定位

当前项目已经完成了一个面向“三峡库区移民安置区高切坡监测预警业务”的 `Text-to-SQL` 原型系统，并在此基础上进一步扩展为一个“最小可用 agent 版本”。

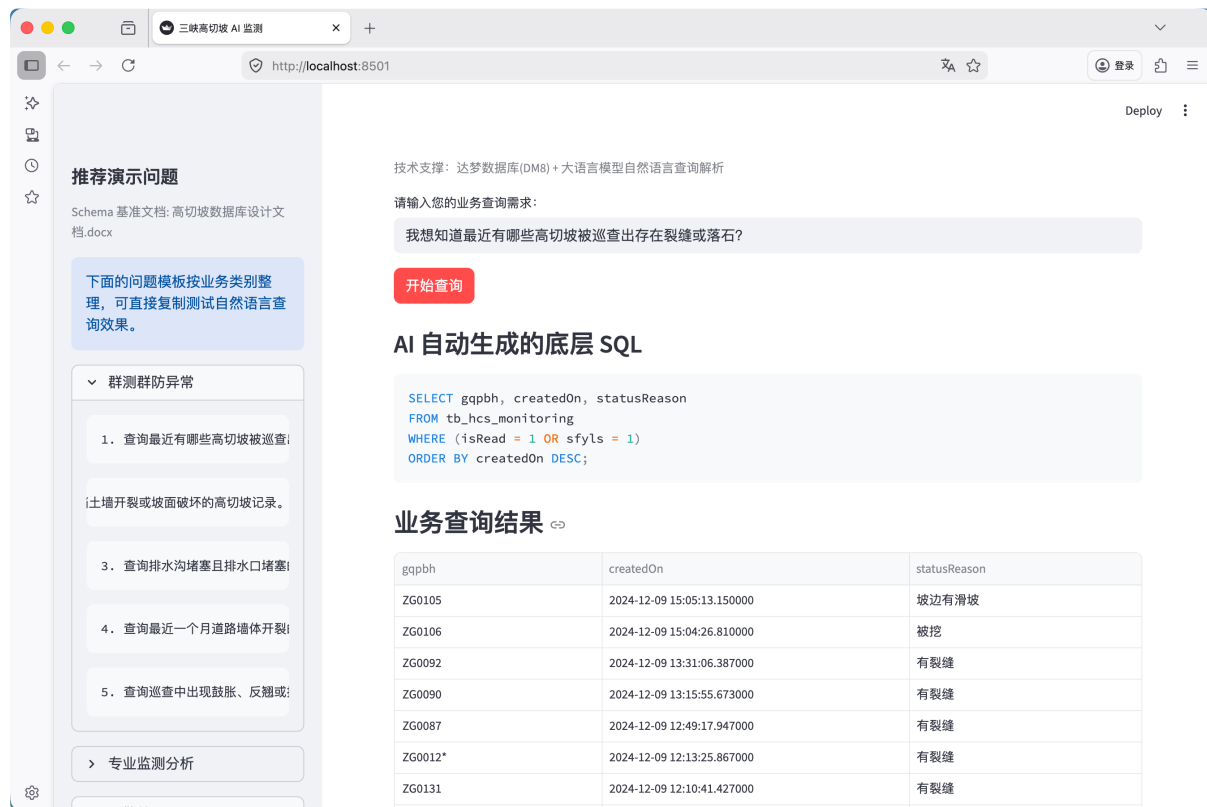
系统目标是让用户直接输入自然语言业务问题，由大语言模型结合高切坡领域知识自动生成达梦数据库

SQL，执行后返回查询结果，并在 SQL 出错时进行有限次数的自动修正。

当前已实现能力

1. Text-to-SQL

- 支持用户输入中文自然语言业务问题。
- 系统可自动生成面向 DM8 的 SQL 查询语句。
- 支持基础单表查询和部分多表关联查询。



2. 网页端展示

- 已基于 Streamlit 提供可运行的网页端界面。
- 页面支持输入业务问题、点击查询、展示底层 SQL 和查询结果。
- 侧边栏已提供分类整理的业务问题模板，便于演示和回归测试。

3. 领域知识注入

- 已结合 [高切坡数据库设计文档.docx](#) 提取表结构摘要。
- 已沉淀高切坡业务语义词典、字段映射、领域规则和典型问法模板。
- 模型已经能够理解“群测群防、专业监测、预警处置、影响人口、裂缝、落石、挡墙开裂、位移”等核心业务概念。

4. SQL 自纠错能力

- 系统已从单次 SQL 执行模式升级为最小可用 agent 模式。
- 当 SQL 执行失败时，agent 会读取数据库报错信息，并尝试自动修正。
- 当前已支持的修正方向包括：
 - 表名错误修正
 - 字段名错误修正
 - 真实数据库与设计文档不一致时的降级修正
 - 预警状态字段字符型/数值型比较错误的启发式修正



5. 真实数据库对象探测

- 系统已支持读取真实数据库中的表和字段信息。
- 探测范围已从 `USER_TABLES` 扩展到 `ALL_TABLES / ALL_TAB_COLUMNS`。
- 当设计文档中存在的表在真实数据库中不存在时，agent 会尽量改用当前可访问的对象。

6. 调试与可解释性

- 页面中已经支持展示 agent 的多轮修正过程。
- 当查询失败时，用户可以看到每次尝试的 SQL 和报错信息。
- 这使系统具备了基本的“可诊断、可定位、可迭代”能力。

当前系统结构

当前项目已从单文件脚本重构为模块化结构，主要包括：

- [app.py](#)
 - Streamlit 推荐入口
- [gqp_ai/ui.py](#)
 - 页面展示逻辑
- [gqp_ai/prompts.py](#)
 - SQL 生成与修复 Prompt
- [gqp_ai/domain_knowledge.py](#)
 - 高切坡领域知识
- [gqp_ai/schema_repository.py](#)
 - 从设计文档提取 schema
- [gqp_ai/database.py](#)
 - 达梦数据库访问与真实 schema 探测
- [gqp_ai/agent.py](#)
 - SQL 自纠错 agent 核心逻辑
- [gqp_ai/services.py](#)
 - 查询编排服务层

当前已知限制

虽然项目已经具备初步可用性，但当前仍属于“原型系统”而非成熟产品，主要限制如下：

1. 真实数据库与设计文档不完全一致

- 某些表在设计文档中存在，但真实数据库中并不存在。
- 某些字段名、字段类型或字段取值与设计文档存在偏差。
- 这会导致模型生成的 SQL 在执行时需要依赖 agent 纠错。

2. 复杂业务问题仍可能失败

- 对复杂的多表关联、模糊业务语义、隐式条件推断问题，系统仍可能无法稳定修正成功。
- 当前 agent 只具备“有限轮数的自纠错”，不是全自动稳定求解器。

3. 预警类字段取值仍有不确定性

- 例如“是否解除预警”“持续预警”等状态字段，真实库中可能是字符串，而不是统一的 `0/1`。

- 当前系统已具备基础启发式修复，但还不能完全覆盖所有实际取值情况。

4. 缺少系统化测试

- 目前已有模板库和手工回归方式，但尚未建立自动化测试集。
- 尚未形成“问题 -> 期望 SQL 特征 -> 期望结果”的标准回归体系。

5. 仍缺少成熟的失败分级与降级响应

- 当前能展示尝试过程，但失败后的用户提示还不够业务化。
- 后续还可以进一步区分：
 - 表不存在
 - 字段不存在
 - 权限不足
 - 数据源缺失
 - 当前问题超出数据库可回答范围

当前阶段结论

综上，当前项目已经：

- 初步完成了 `Text-to-SQL`
- 完成了网页端交互展示
- 初步完成了高切坡领域知识注入
- 初步具备了 SQL 自动纠错和多轮修正能力

因此，当前项目可以被定义为：

一个“可运行、可演示、具备基础自纠错能力的高切坡监测业务 `Text-to-SQL` 原型系统”

或者更准确地说：

一个“最小可用 agent 版本”的高切坡数据库自然语言查询系统

下一阶段建议

如果继续迭代，建议按以下优先级推进：

1. 增强 agent 的查询与纠错能力

- 将当前“最小可用 agent”继续升级为“面向数据库问答的业务 agent”。
- 强化错误分类处理能力，重点覆盖：
 - 表不存在

- 字段不存在
 - 字段类型不匹配
 - owner/schema 不匹配
 - 权限不足
 - 无法回答时的降级响应
- 让 agent 在多轮修复之外，具备“失败归因 + 替代查询建议 + 可回答范围提示”能力。

2. 建设 Schema RAG 知识库

- 将 [高切坡数据库设计文档.docx](#) 进一步拆解为可检索的知识片段。
- 建立以“表级、字段级、关系级、规则级”为粒度的 `Schema RAG`。
- 让 agent 在生成 SQL 和修复 SQL 时，优先检索 Schema RAG，而不是只依赖长 prompt。
- 需要明确一个原则：
 - `真实数据库 schema` 决定“能不能查、怎么查”
 - `Schema RAG` 决定“字段和表在业务上是什么意思”

3. 建设 Domain RAG 领域知识库

- 将年度技术报告、年度工作报告、预警经验、巡查规律等资料做成可检索知识库。
- 重点沉淀以下知识：
 - 高切坡异常现象及其业务含义
 - 群测群防与专业监测的判读规则
 - 汛期、降雨、排水堵塞、工程老化、人类工程活动等诱因分析
 - 持续预警、解除预警、缓慢变形等业务概念
- 这个知识库不替代数据库，而是用于增强“查到结果后的专业解释”能力。

4. 让 agent 能结合查询结果做专业分析

- 让系统不只返回“SQL 结果表格”，还能够返回“结果解释”。
- 推荐后续形成两阶段处理链路：
 - a. `SQL Agent` 负责查询真实数据库
 - b. `Analysis Agent` 负责结合查询结果 + Domain RAG 进行业务分析
- 重点支持的分析输出包括：
 - 查询结果摘要
 - 异常点解释
 - 可能诱因分析
 - 风险关注建议
 - 是否建议进一步复核或加密监测

- 这一步是项目从“数据库查询工具”走向“专业辅助分析系统”的关键。

5. 建立标准化测试集与评估体系

- 基于现有模板库，继续沉淀一批高频业务问题。
- 为每个问题定义：
 - 命中表
 - 关键字段
 - 允许的 SQL 结构
 - 预期结果特征
 - 是否应触发业务分析
- 将测试分为两类：
 - Text-to-SQL 正确性测试
 - 查询结果专业分析测试

6. 最后再推进生产化能力

- 在 agent 和 RAG 路线基本稳定后，再进一步补强：
 - 日志记录
 - 配置管理
 - 异常分级
 - 权限和安全限制
 - 部署与运维方案
- 生产化阶段应以“先稳定能力，再增强工程”作为顺序。

结语

目前项目已经跨过了“单纯演示脚本”的阶段，具备了原型系统的基本形态，也已经开始体现 agent 化能力。后续工作的重点，不再是“能不能生成 SQL”，而是“怎样让 SQL 更稳定、更贴近真实业务、更能适配真实数据库差异”。